

EMBEDDED MACHINE LEARNING · PHD COURSE

SESSION 03

Classical versus deep learning — and how to make models small

A primer on learning, the two model families and their inference cost, the architectures designed for small devices, and the three ways to shrink a trained network.

THE QUESTION OF THE SESSION

A model is a parameterised function fitted to examples — and, on a device, a bill in bytes and operations.

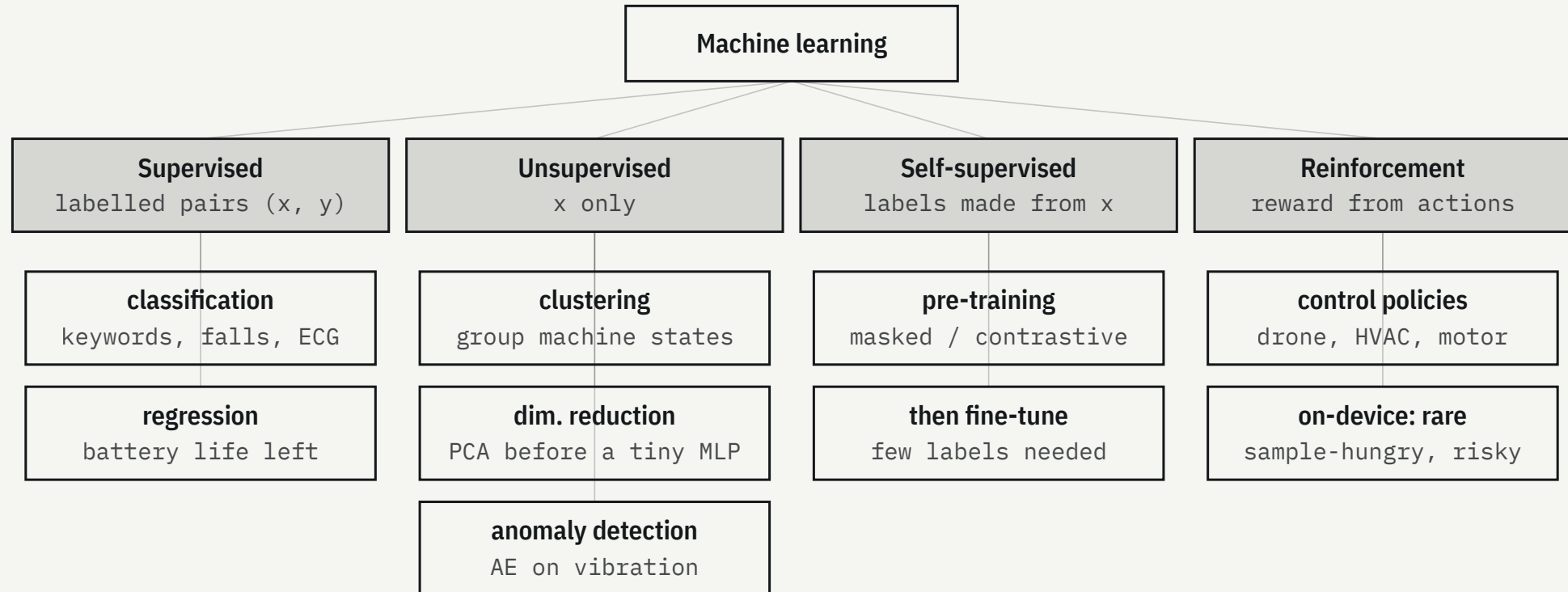
01

A machine-learning primer

Loss, data splits, gradient descent, evaluation — the minimum to read the rest of the course.

PRIMER

The families of machine learning



Almost every deployed embedded model today is supervised classification;
anomaly detection is the main unsupervised exception.

PRIMER · LEARNING AS OPTIMISATION

Training minimises a loss over examples

$$\theta^* = \operatorname{argmin}_{\theta} \left(\frac{1}{N} \sum_i \ell(\mathbf{f}(\mathbf{x}_i; \theta), y_i) + \lambda \cdot R(\theta) \right)$$

regression $\ell = (\hat{y} - y)^2$

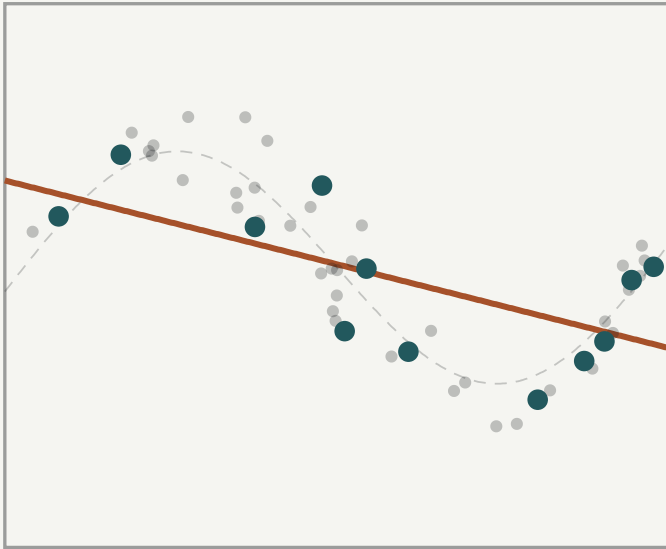
classification $p^c = e^{\{z^c\}} / \sum_k e^{\{z_k\}}$ $\ell = -\log p_y$ (cross-entropy)

The first term is the average training loss; the regulariser R (e.g. weight decay $\|\theta\|^2$) prefers simpler parameters, with strength λ chosen by the engineer — a hyper-parameter.

PRIMER · GENERALISATION

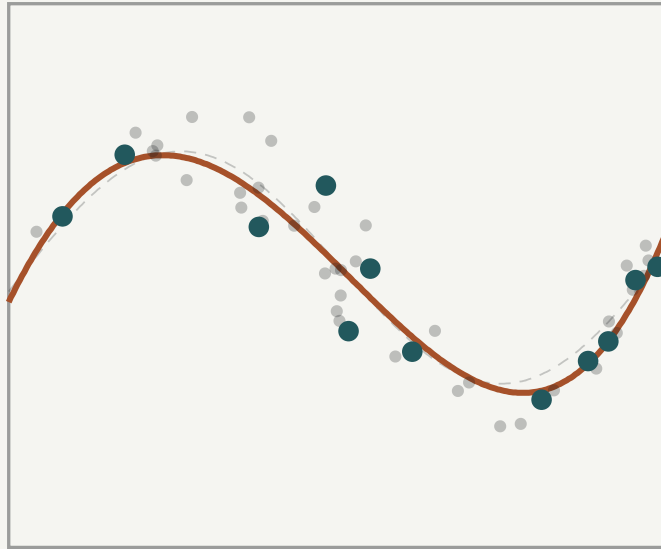
Training error falls; test error does not

degree 1 — underfit



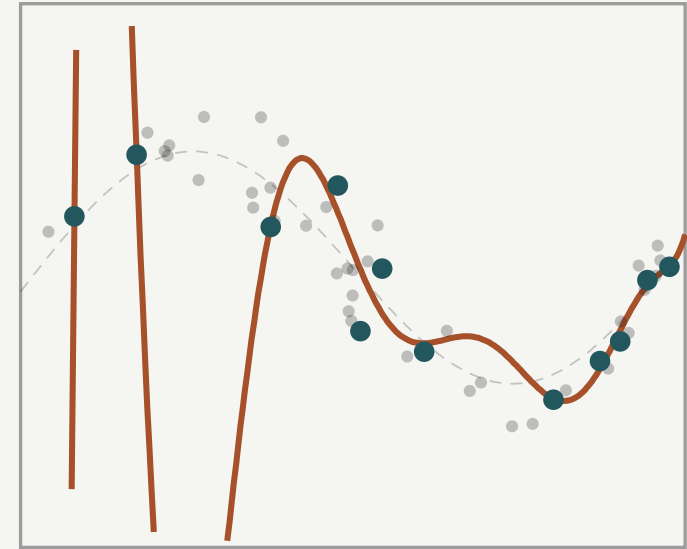
train error 0.118
test error 0.173

degree 4 — about right



train error 0.034
test error 0.035

degree 9 — overfit



train error 0.022
test error > 10 (off the chart)

teal: 12 training points · faint: 40 test points · dashed: the true function · copper: the fitted model

PRIMER · HONEST EVALUATION

Fit on train, choose on validation, report test once

Hold-out split

A horizontal bar divided into three segments: a dark teal segment on the left labeled 'train 70 %', an orange segment in the middle labeled 'validation 15 %', and a light gray segment on the right labeled 'test 15 %'.

train 70 % validation 15 % test 15 %

fit parameters on train · choose hyper-parameters on validation · touch test once, at the end

5-fold cross-validation



every example is validated exactly once; report mean $\pm$ spread over folds

Grouped (subject-wise) split — mandatory for windowed sensor data



train: subjects A-D

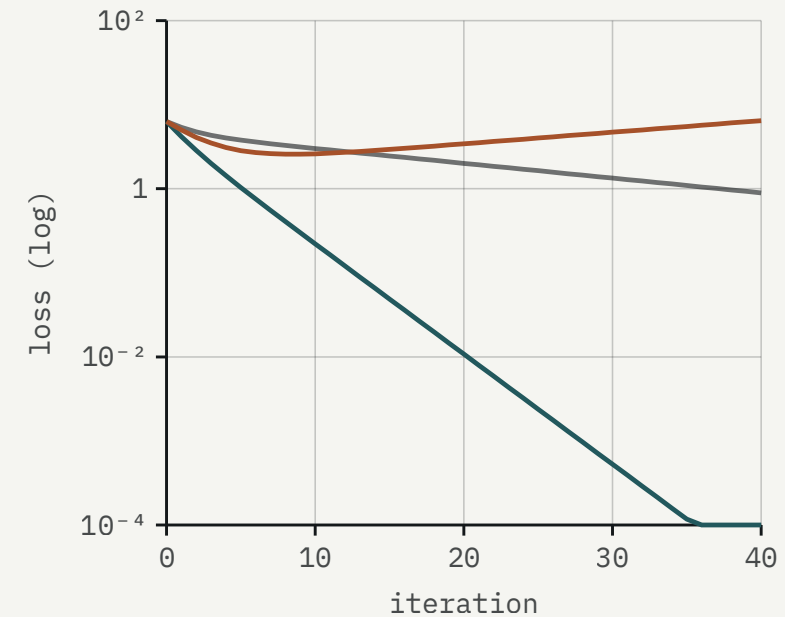
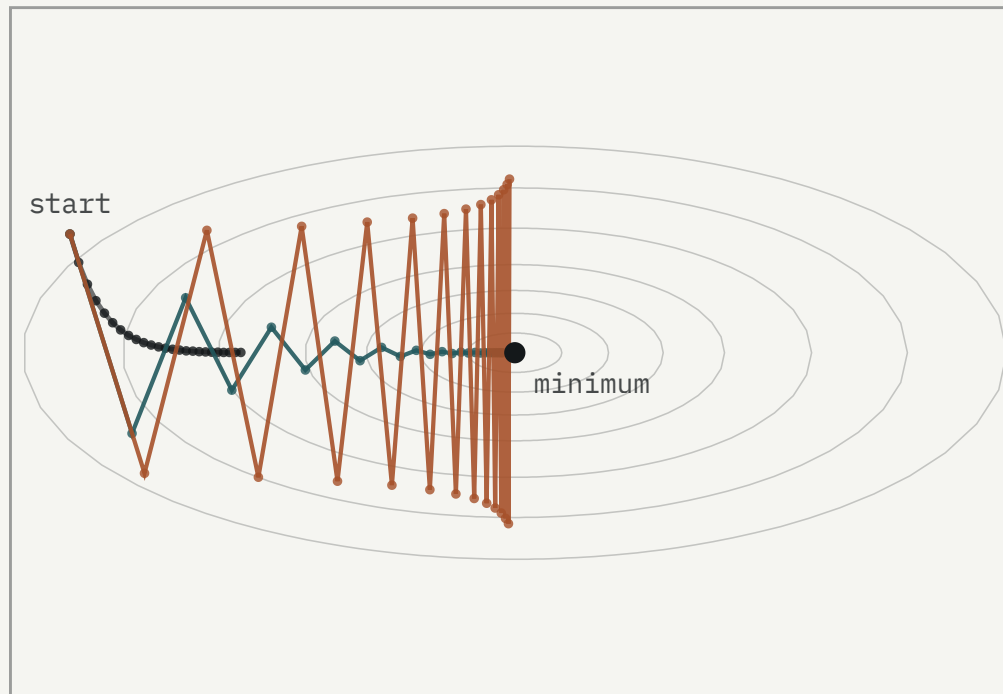
test: subjects E-F

Overlapping windows from one recording are near-duplicates: a random split leaks them across the line.

PRIMER · OPTIMISATION

Gradient descent, and why feature scaling matters

loss contours: steep in w_2 , shallow in w_1



- $\eta = 0.02$ too small
- $\eta = 0.14$ good
- $\eta = 0.168$ too large

PRIMER · EVALUATION

Accuracy hides rare events

		predicted	
		event	no event
actual	event	40 TP true positive	10 FN missed event
	no event	60 FP false alarm	890 TN true negative

recall (sensitivity)

$$TP / (TP + FN)$$

$$40 / 50 = 0.80$$

precision

$$TP / (TP + FP)$$

$$40 / 100 = 0.40$$

F1

$$2PR / (P + R)$$

$$0.53$$

specificity

$$TN / (TN + FP)$$

$$890 / 950 = 0.94$$

accuracy

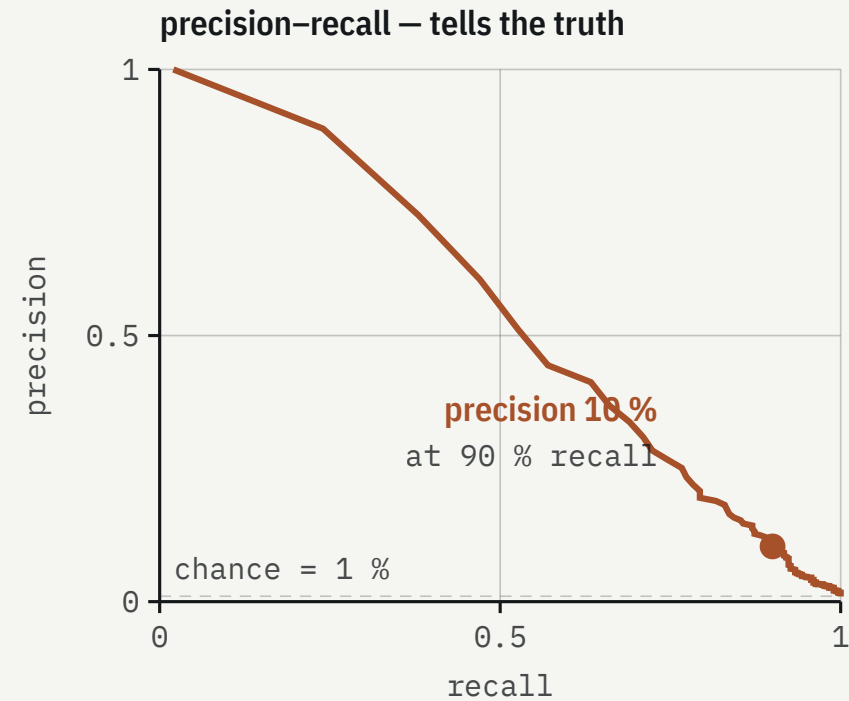
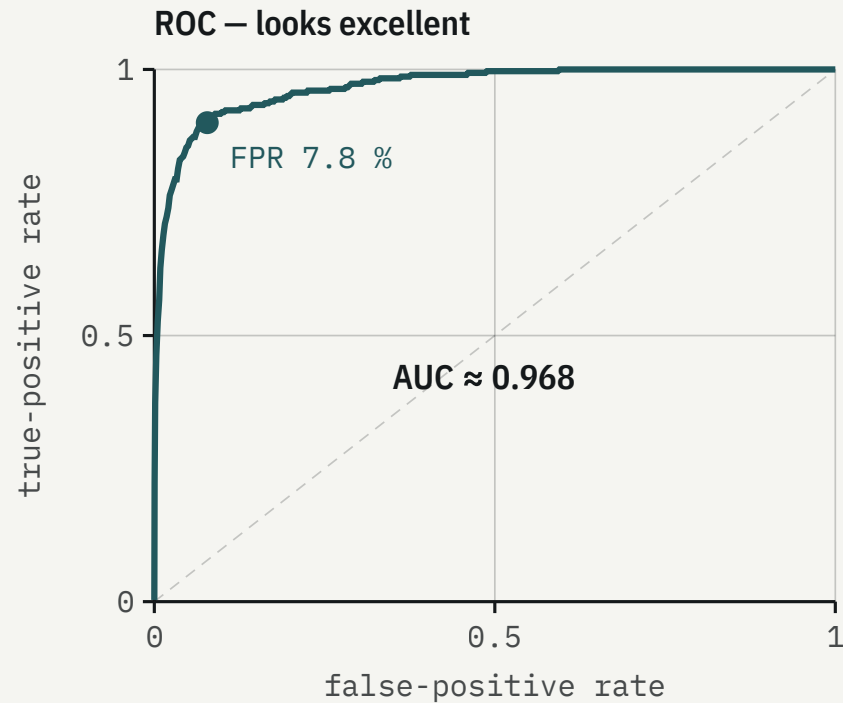
$$(TP + TN) / \text{all}$$

$$930 / 1000 = 0.93$$

A detector that always says "no event" scores
accuracy 0.95 – better than this one, and useless.

THE METRIC MISTAKE THAT SHIPS PRODUCTS

Same detector. One panel lies.



At 90 % recall this detector raises about **nine false alarms for every true one** – with an AUC of 0.97 and 92 % accuracy.

02

Classical models under constraints

When a non-neural model still wins — and what each family costs at inference.

THE THESIS

Choosing a model family is choosing which shapes are cheap to express — and which dimension your inference cost is allowed to grow with.

THE REGIME QUESTION

Classical wins when at least one of these holds

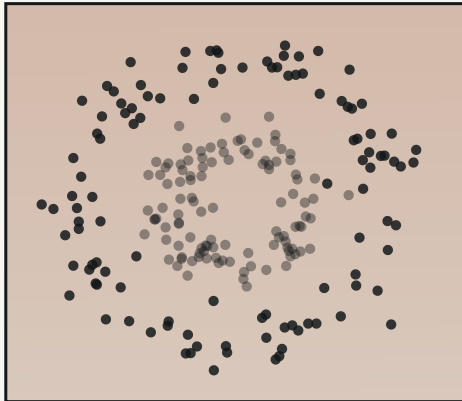
- **Little data.** A few hundred labelled windows: a regularised linear model or a small forest generalises better than a network you cannot regularise into submission.
- **Extreme memory limits.** Thirty depth-6 boosted trees fit in a few kilobytes of flash and need almost no RAM.
- **Deterministic latency.** A tree's latency varies with the path; a linear model's does not vary at all.
- **Interpretability or certification.** Medical and automotive regimes are far happier with a model you can print as a rule.
- **Good features already exist.** If domain physics gives you the right twenty features, the representation-learning advantage evaporates.

They lose when the input is high-dimensional and the useful features are unknown.

INDUCTIVE BIAS MADE VISIBLE

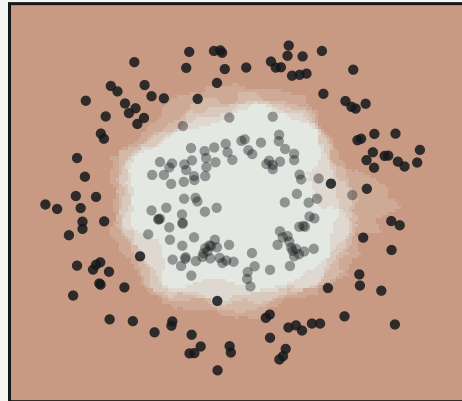
Same data, same features, four different biases

logistic regression



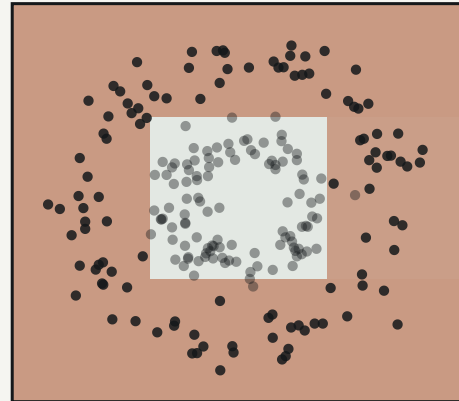
train accuracy 55 %

k-NN, k = 5



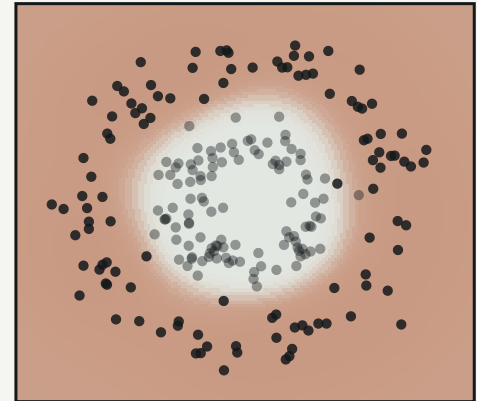
train accuracy 98 %

decision tree, depth 4



train accuracy 100 %

RBF SVM



train accuracy 99 %

Same data, same features, four different inductive biases.

The tree's boxes are cheap at inference – a handful of comparisons – but they can only ever be boxes.

The RBF SVM draws the right shape and pays for it with stored support vectors.

Logistic regression cannot separate these classes at all – until you add the feature x^2+y^2 , which is the entire point of the kernel trick.

THE SPINE OF THE SESSION

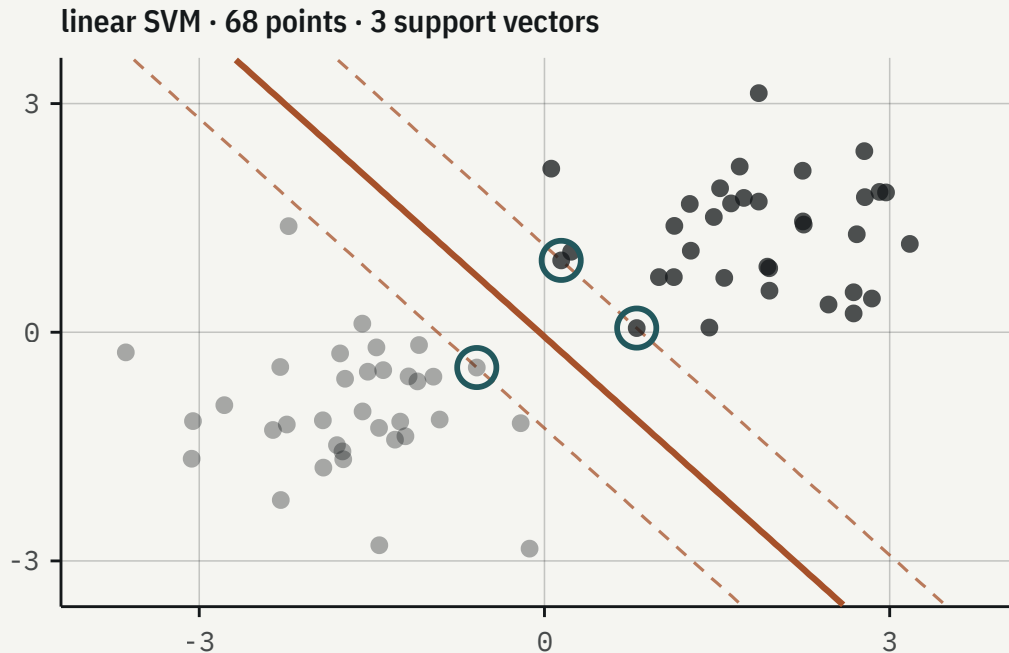
Inference cost, by family

MODEL	ARITHMETIC	PARAMETERS	RAM	REMARK
Logistic regression	$C \cdot D$ MACs	$C(D+1)$	$O(C)$	The reference baseline
Gaussian naive Bayes	$C \cdot D$	$2CD$	$O(C)$	Robust with tiny data
k-NN, brute force	$N \cdot D$ MACs	$N \cdot D$	$O(k)$	No training; prohibitive memory
Linear SVM	D MACs per binary classifier	$D+1$	$O(1)$	One-vs-rest: same cost as logistic, different loss
Kernel SVM (RBF)	$n_{SV} \cdot D + n_{SV} \exp$	$n_{SV} \cdot D$	$O(n_{SV})$	Grows with the dataset
Decision tree, depth d	d compares	$\sim 2^d$ nodes	$O(1)$	Compiles to nested <i>if</i>
Random forest, T trees	$T \cdot d$ compares	$T \cdot 2^d$	$O(1)$	Thresholds quantize well
Boosted stumps, T rounds	T compares + T adds	$\sim 3T$	$O(1)$	Best accuracy-per-byte on tabular features

Two of these cost **comparisons**, not multiplications – so a MAC accelerator does nothing for them.

FITTED, NOT SKETCHED

Three of sixty-eight points carry the whole decision function



$$\text{margin} = 2/\|w\|$$

Only the circled points enter the solution. Delete any other point and the boundary does not move.

Inference cost:

linear kernel $\rightarrow$ D MACs

RBF kernel $\rightarrow n_{SV} \cdot$ D MACs

and n_{SV} grows with the size of the training set.

Delete any non-circled point and the boundary does not move. That sparsity is the beautiful part – and the fatal part.

WHY THIS MATTERS ON A MICROCONTROLLER

A boosted ensemble compiles into straight-line comparison code with the thresholds as immediates.

No matrix, no accumulator, no arena. Tens of comparisons and adds, often under a microsecond, with a completely flat memory profile. For tabular sensor features this is frequently both the most accurate and the cheapest option.

03

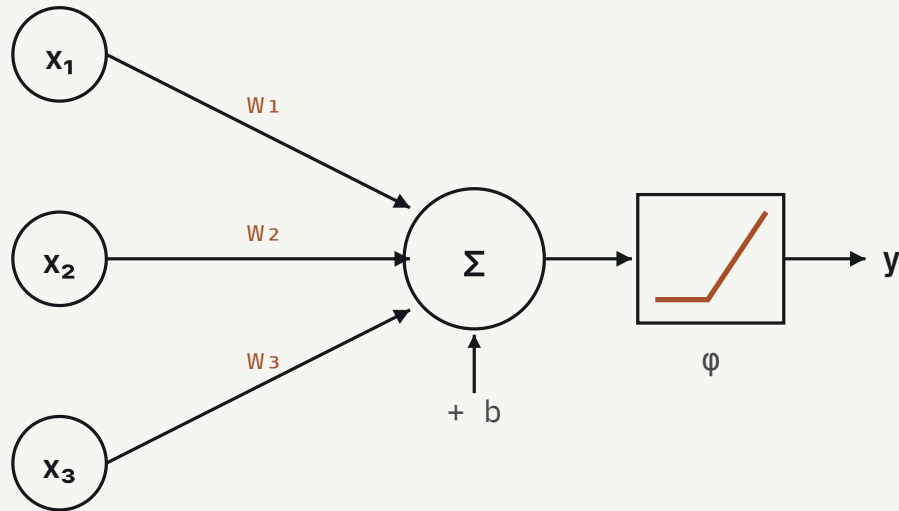
Neural networks, from one neuron

Count the cost at every step.

PRIMER · NEURONS

A neuron, a layer, a network

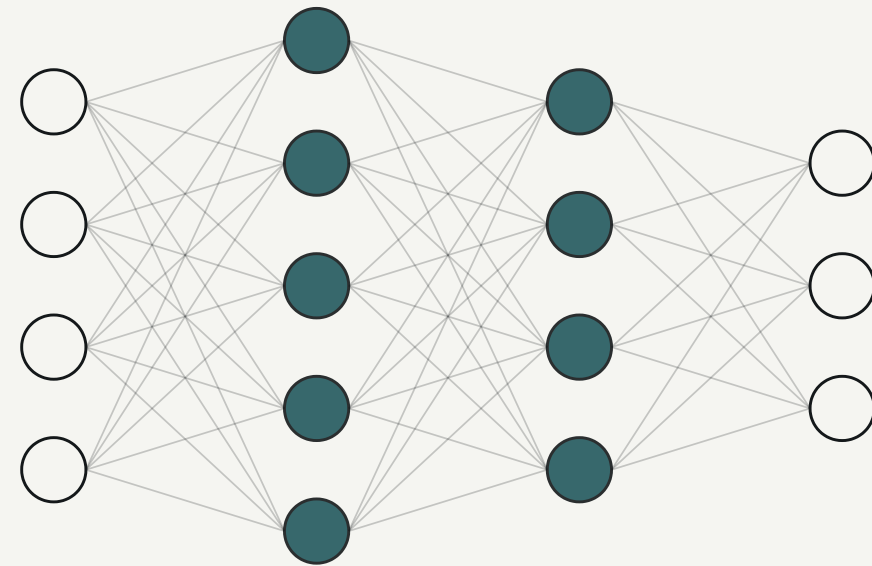
one neuron: $y = \varphi(w \cdot x + b)$



cost: n MACs + 1 add + 1 activation

parameters: n weights + 1 bias

multilayer perceptron (fully connected)



input

hidden 1

hidden 2

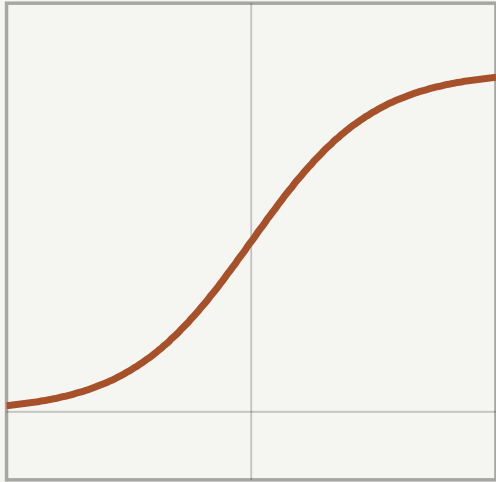
output

params = $(4 \cdot 5 + 5) + (5 \cdot 4 + 4) + (4 \cdot 3 + 3) = 64$ · MACs = 52

PRIMER · ACTIVATIONS

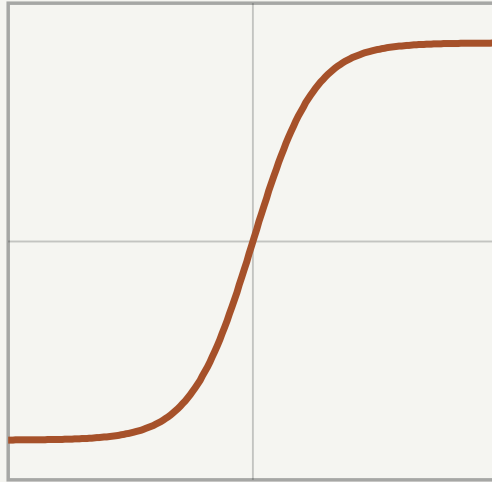
The non-linearity, and what it costs on integer hardware

sigmoid



exp → lookup table

tanh



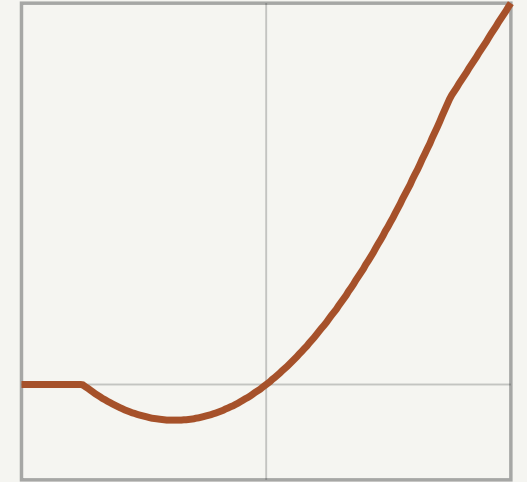
exp → lookup table

ReLU



one compare: free

hard-swish



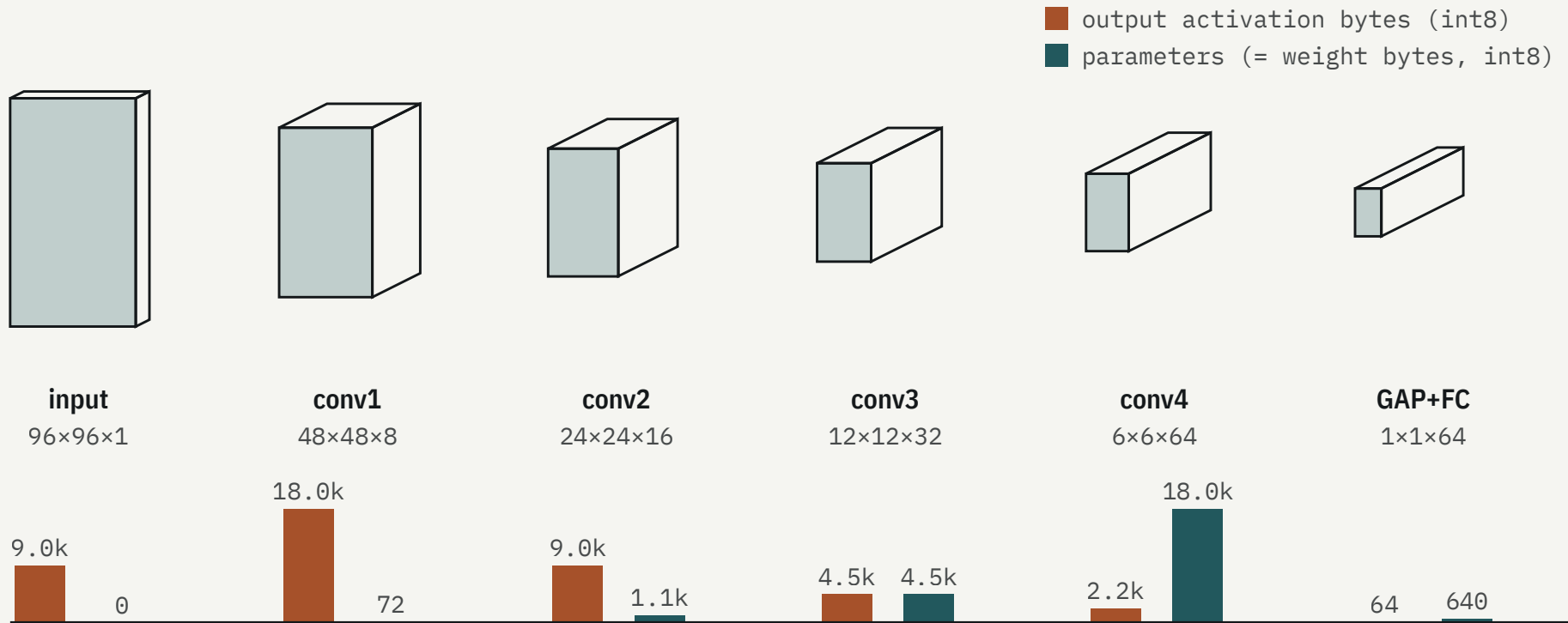
piecewise linear: cheap

Element-wise. ReLU and ReLU6 fold into the clamp that ends an int8 layer.

CONVOLUTIONAL NETWORKS

Space shrinks, channels grow – activations front, parameters back

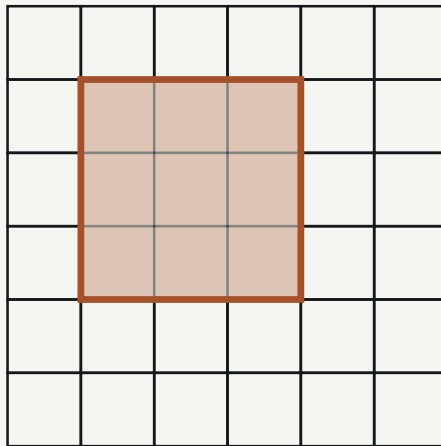
stride-2 3×3 convolutions: space shrinks 2× per stage, channels double



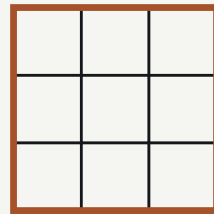
activations peak at the front, parameters at the back – the asymmetry that decides SRAM versus flash

READ THREE COSTS OFF ONE PICTURE

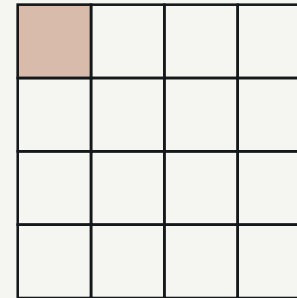
What a convolution actually costs

input $6 \times 6 \times C_{in}$ 

each window: $k^2 \cdot C_{in}$
multiply-accumulates

kernel $k=3$ 

one kernel per
output channel

output $4 \times 4 \times C_{out}$ 16 windows $\times C_{out}$

MACs =
 $H \cdot W \cdot k^2 \cdot C_{in} \cdot C_{out}$

params =
 $k^2 \cdot C_{in} \cdot C_{out}$

activations =
 $H \cdot W \cdot C_{out}$

Every convolution is this picture with different numbers

$$\text{MACs} = H \cdot W \cdot k^2 \cdot C_{in} \cdot C_{out} \quad \cdot \quad \text{params} = k^2 \cdot C_{in} \cdot C_{out} \quad \cdot \quad \text{activations} = H \cdot W \cdot C_{out}$$

LEARN THESE

Layer cost formulas

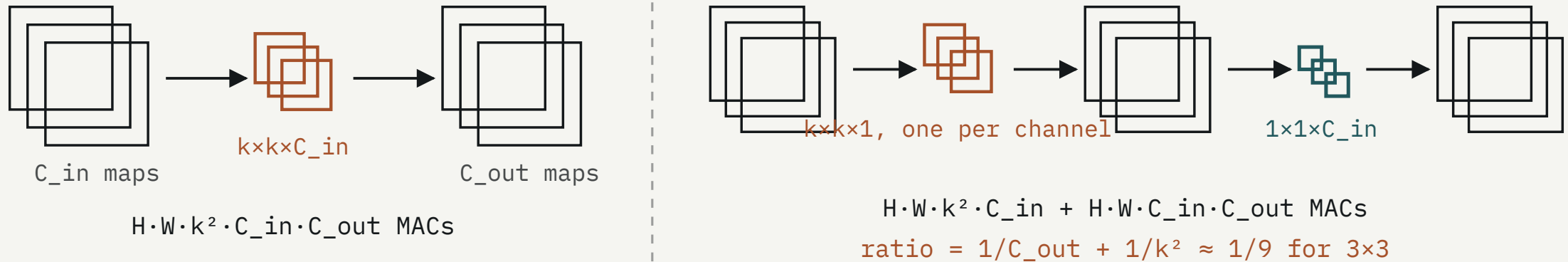
LAYER	PARAMETERS	MACS	OUTPUT ACTIVATIONS
Dense, $D_{in} \rightarrow D_{out}$	$D_{in} \cdot D_{out} + D_{out}$	$D_{in} \cdot D_{out}$	D_{out}
Conv 2-D, $k \times k$, $C_{in} \rightarrow C_{out}$	$k^2 \cdot C_{in} \cdot C_{out} + C_{out}$	$H \cdot W \cdot k^2 \cdot C_{in} \cdot C_{out}$	$H \cdot W \cdot C_{out}$
Depthwise conv, $k \times k$	$k^2 \cdot C_{in}$	$H \cdot W \cdot k^2 \cdot C_{in}$	$H \cdot W \cdot C_{in}$
Pointwise 1×1	$C_{in} \cdot C_{out}$	$H \cdot W \cdot C_{in} \cdot C_{out}$	$H \cdot W \cdot C_{out}$
Grouped conv, g groups	$k^2 \cdot C_{in} \cdot C_{out} / g$	$H \cdot W \cdot k^2 \cdot C_{in} \cdot C_{out} / g$	$H \cdot W \cdot C_{out}$
Dilated conv, rate r	same as conv	same as conv	same as conv
Batch norm (inference)	$2C$ stored	0 after folding	in place
Residual add	0	$H \cdot W \cdot C$ adds	holds the skip tensor live

Learn these; the rest of the course assumes them. Note the dilated row: **larger receptive field, identical cost.**

DERIVATION

One operation split into two cheap ones

standard 3×3 conv – one kernel spans all inputs depthwise separable – two cheaper stages



The arithmetic falls by ~9×. The bytes moved fall by far less – see the roofline.

this gap between counted work and measured time is the central lesson of the session

$$\text{ratio} = (k^2 \cdot C_{in} + C_{in} \cdot C_{out}) / (k^2 \cdot C_{in} \cdot C_{out}) = \mathbf{1/C_{out} + 1/k^2} \approx 1/9 \text{ for a } 3 \times 3 \text{ kernel.}$$

FOLDING, AND WHY IT IS MANDATORY

Batch norm disappears into the convolution

$$y = \gamma (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta \quad \text{with} \quad x = W * a + b$$

$$\Rightarrow y = W' * a + b'$$

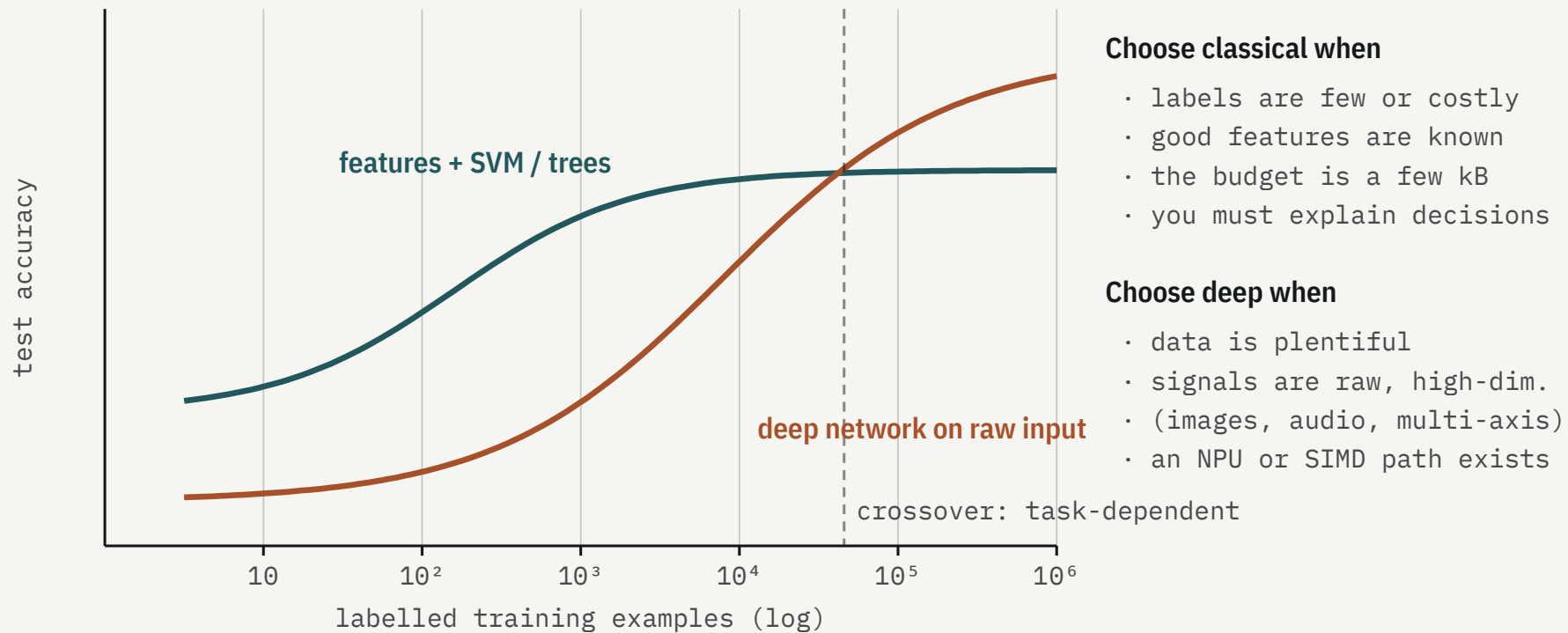
$$W' = \gamma W / \sqrt{(\sigma^2 + \epsilon)}$$

$$b' = \gamma(b - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$$

Zero inference cost and one fewer pass over the activations. The fold is also a PRECONDITION for quantization — an unfolded BN would need a floating-point per-channel scale at runtime, and the fold is what lets per-channel weight scales absorb it instead.

DECISION GUIDE

Classical or deep?



Schematic, not measured: the shape is typical, the numbers are not.

BREAK

Five minutes.

Next: efficient architectures, then compression.

04

Efficient architectures

Designing the network to be cheap from the start —
presentation topic C.

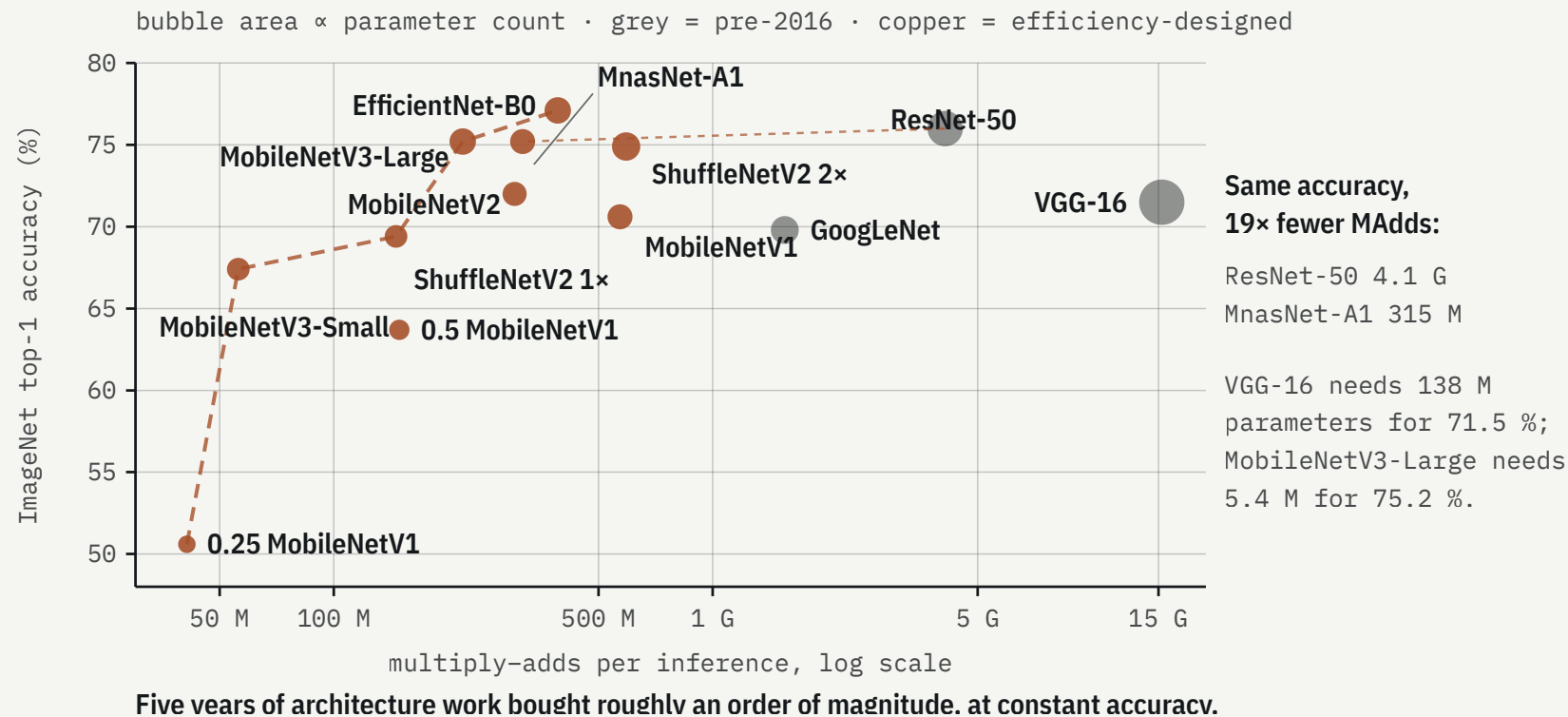
THE THESIS

Every efficient architecture is a factorisation plus an assumption.

The design work is choosing a factorisation whose assumption survives on *your* hardware. Search is a way of automating that choice against a measured cost model.

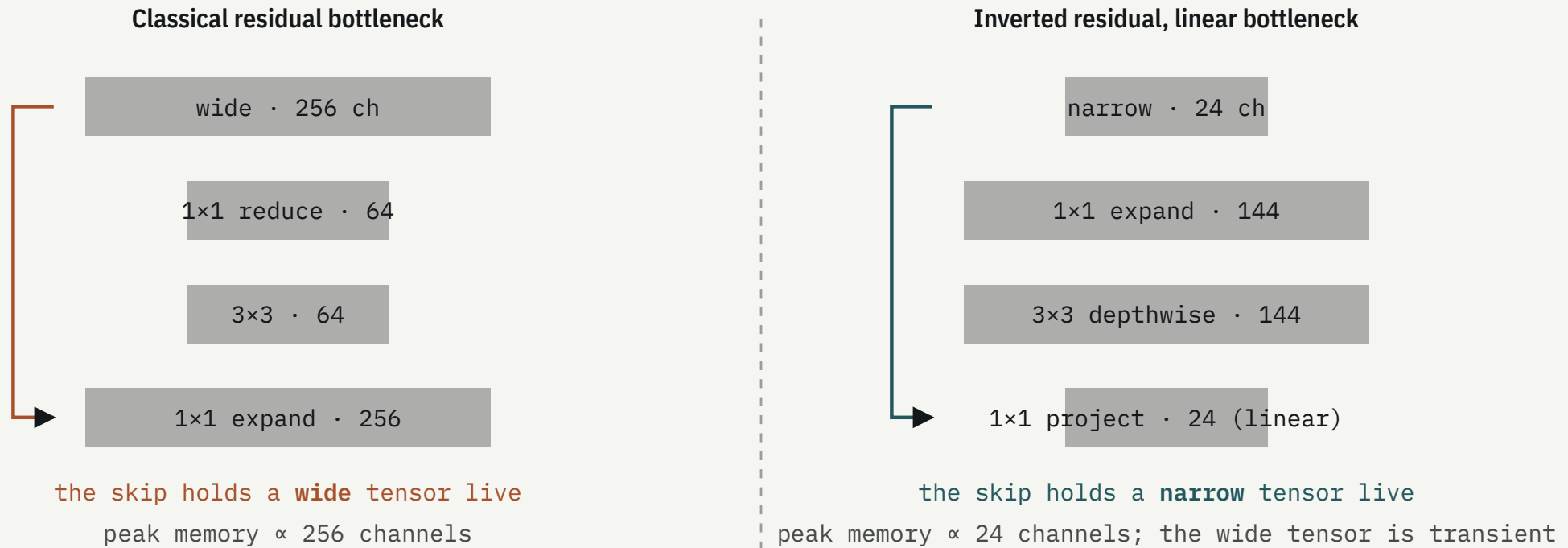
PUBLISHED NUMBERS

Five years moved the frontier an order of magnitude



A MEMORY ARGUMENT, NOT AN ACCURACY ARGUMENT

Why the residual was inverted



Inversion is a memory argument, not an accuracy argument.

The final projection is **linear** – no ReLU – because a ReLU in a low-dimensional space destroys information that cannot be recovered.

DERIVATION

Where EfficientNet's constraint comes from

cost of scaling one dimension by a factor ϕ

depth $d = \alpha^\phi$

linear in ϕ



each extra layer adds its own MACs

width $w = \beta^\phi$

quadratic



C_{in} and C_{out} both grow

resolution $r = \gamma^\phi$

quadratic



H and W both grow

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

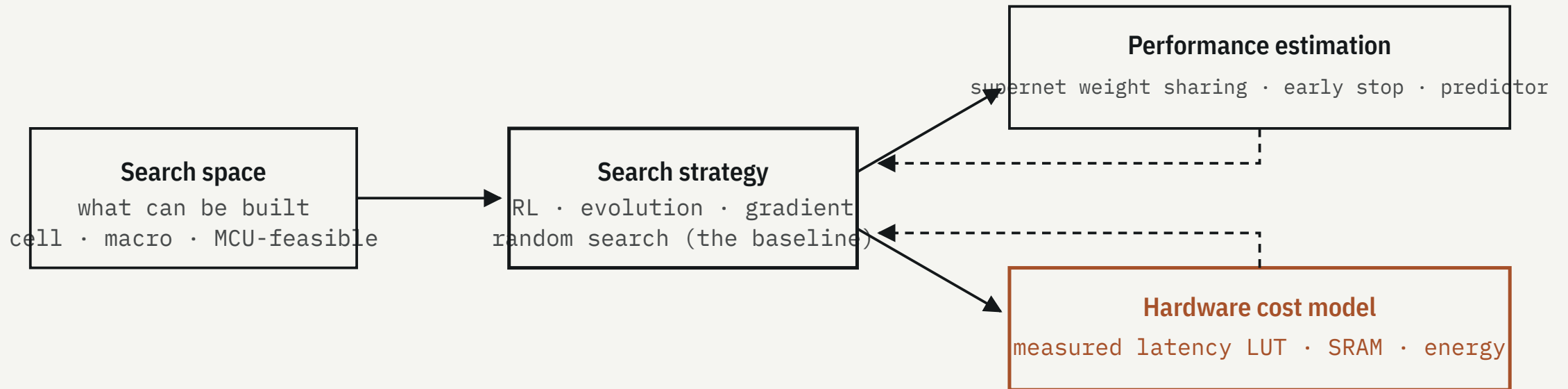
One unit of ϕ therefore doubles the FLOPs, whichever mix of the three you choose.

Note what is being held constant: FLOPs — not SRAM. Raising resolution is far more expensive on an MCU.

$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ — because depth enters the cost linearly while width and resolution enter **quadratically**.

DECOMPOSED

Search space, strategy, estimation – and hardware cost



The hardware box is what turns NAS into embedded ML.

Without it you get an accurate model; with it you get a deployable one.

05

Compression

Fewer bits, fewer numbers, a smaller student — presentation topics A and B.

THE TOOLKIT

Four families, one question: can the hardware cash it in?

FAMILY	WHAT SHRINKS	TYPICAL GAIN	PAYS OFF WHEN
Quantization	bits per weight and activation	4× memory; 2–4× speed on int8	almost always — the default
Pruning	weights, or whole channels	2–10× weights; 1.5–3× compute	structured: always; unstructured: only with sparse support
Distillation	the model — a student replaces the teacher	whatever the student allows	a good teacher and a training budget
Low rank / sharing	rank; distinct values	2–5× on large dense layers	large dense layers; flash-bound

THE WHOLE IDEA IN ONE SENTENCE

Quantization replaces a per-value exponent with one scale shared by a whole tensor.

That is why it is so cheap in silicon — and exactly why it breaks whenever the values in a tensor are not of comparable size.

FOUNDATIONS

Two ways to spend bits

float32 — 32 bits per value: sign · exponent · mantissa



bfloat16 — same exponent range, coarser steps



int8 affine — 8 bits per value



float: each value carries its own exponent → wide dynamic range, per-value

int8: one shared scale → uniform steps, and the whole scheme works only

because values inside one tensor have similar magnitudes.

When they do not — depthwise channels, transformer activation outliers — quantization fails.

An int8 multiply costs roughly a **twentieth** of an fp32 one — before counting the halved memory traffic.

DERIVATION

Scale and zero-point from a range

$$x \approx s \cdot (q - z) \quad \text{de-quantization}$$

$$q = \text{clamp}(\text{round}(x/s) + z, Q_{\min}, Q_{\max}) \quad \text{quantization}$$

$$s = (\beta - \alpha) / (Q_{\max} - Q_{\min})$$

$$z = \text{round}(Q_{\min} - \alpha / s)$$

The zero-point z is an INTEGER, and that is not a detail: it guarantees that real zero is represented exactly. Padding introduces zeros everywhere, and an inexact zero would inject a bias at every border and every masked position.

DO IT WITH A NUMBER

Quantize a weight tensor spanning $[-0.62, 0.48]$

1. $s = (0.48 - (-0.62)) / 255 = 1.10 / 255 = 4.3137 \times 10^{-3}$
2. $z = \text{round}(-128 - (-0.62)/s) = \text{round}(-128 + 143.73) = 16$
3. $x = 0.31 \rightarrow q = \text{round}(71.86) + 16 = \mathbf{88}$, $\hat{x} = s(88 - 16) = 0.3106$ — error 5.9×10^{-4}
4. $x = -0.62 \rightarrow q = \text{round}(-143.73) + 16 = \mathbf{-128}$, clamped exactly
5. $x = 0 \rightarrow q = 16$, $\hat{x} = \mathbf{0}$ exactly

Maximum representation error = $s/2 = 2.16 \times 10^{-3}$, uniform across the range.

DERIVATION · JACOB ET AL., CVPR 2018

An inner product with no floating point anywhere

$$s_3(q_3 - z_3) = \sum_j s_1(q_{1j} - z_1) \cdot s_2(q_{2j} - z_2)$$

$$q_3 = z_3 + M \cdot \sum_j (q_{1j} - z_1)(q_{2j} - z_2), \quad M = s_1 s_2 / s_3$$

$$\sum_j (q_{1j} - z_1)(q_{2j} - z_2) = \sum q_1 q_2 - z_2 \sum q_1 - z_1 \sum q_2 + N z_1 z_2$$

↑GEMM ↑row sum ↑folds into bias ↑constant

M is empirically always in (0,1). Write $M = 2^{(-n)} \cdot M_0$ with $M_0 \in [0.5, 1)$ stored as a fixed-point int32: multiplying by M is a widening multiply and a rounding right shift. That is what 'integer-only' means — it runs on a core with no FPU at all.

THE FAILURE EVERYONE MEETS ONCE

Why per-tensor scales destroy depthwise layers

per-channel weight ranges after batch-norm folding, depthwise layer



Per-channel scales: one s per output channel, so every channel uses the full integer grid.

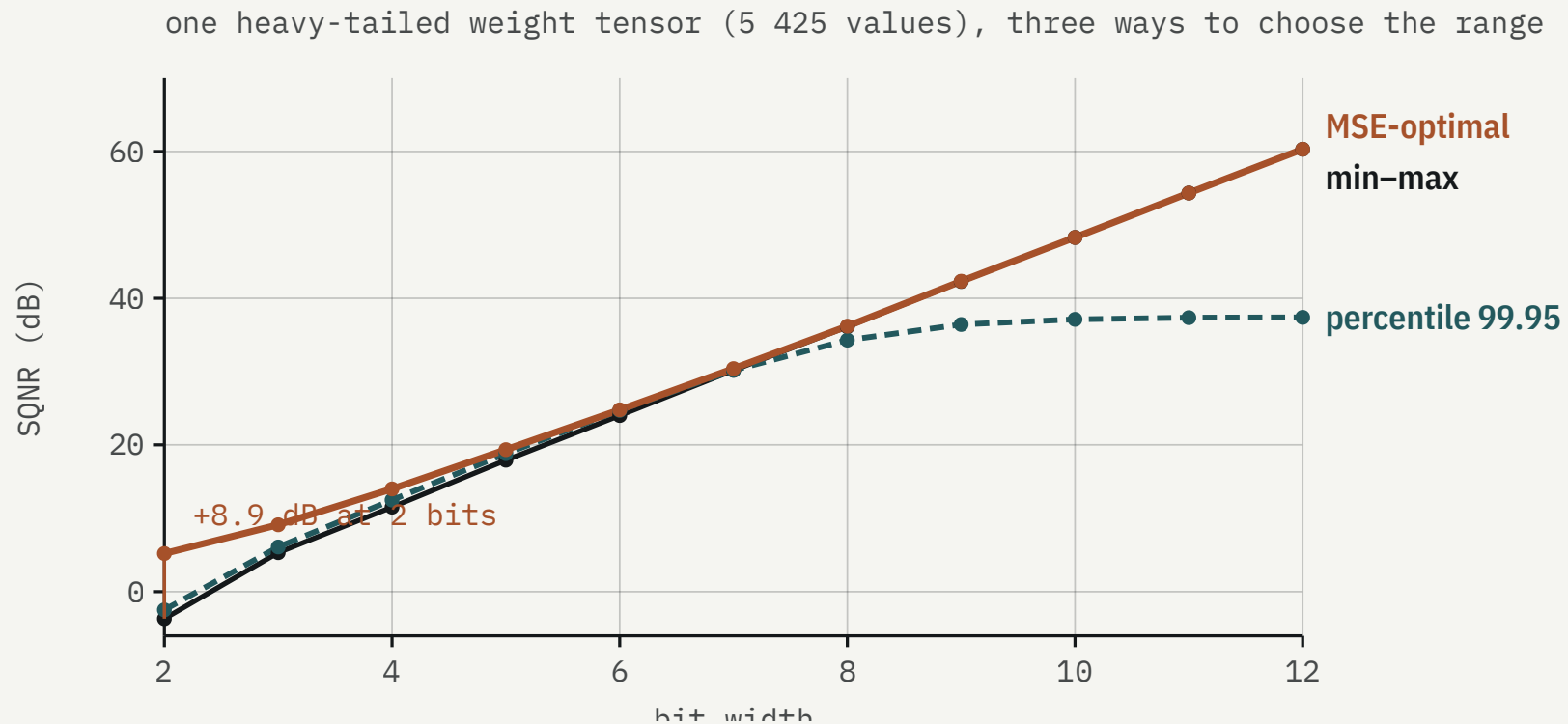
Cost: a vector of requantization multipliers instead of one scalar – a few hundred bytes, no extra arithmetic in the inner loop.

Activations stay per-tensor: their scale must be known before the sum, so a per-channel activation scale would break the shared accumulator.

The fix is **granularity, not more bits**: one scale per output channel, a vector of requantization multipliers, no extra inner-loop arithmetic.

MEASURED ON THAT TENSOR

The right range depends on the bit width



Clip when levels are scarce, keep the tail when they are plentiful – and choosing costs **nothing** at inference.

STE AND LSQ

Getting a gradient through a step function

$$\hat{x} = s \cdot (\text{clamp}(\text{round}(x/s) + z) - z)$$
 fake quantization in the forward pass
$$\partial \hat{x} / \partial x := 1 \quad \text{inside the clipping range, } 0 \text{ outside} \quad (\text{straight-through estimator})$$
$$\partial \hat{x} / \partial s = -x/s + \text{round}(x/s) \quad \text{inside, } Q_{\min} \text{ or } Q_{\max} \text{ outside} \quad (\text{LSQ})$$

The STE is a biased estimator of a gradient that does not exist — and it works, reliably, which is one of the more interesting unexplained facts in the area. LSQ treats the step size as a trained parameter and made 2–4-bit QAT routine rather than heroic.

THE TENSION

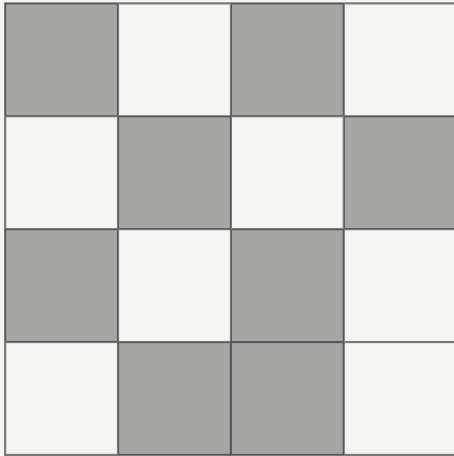
The sparsity pattern that costs the least accuracy is the one your processor cannot exploit.

Everything in this session is a negotiation between those two facts.

TAXONOMY

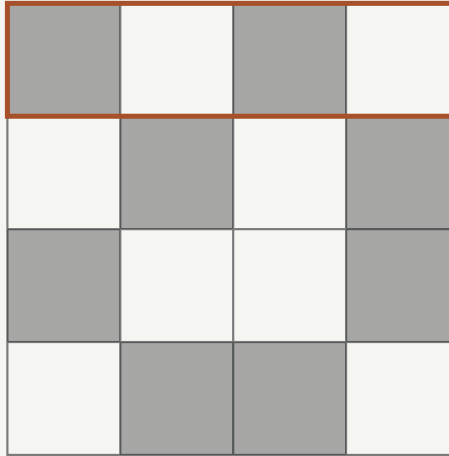
Three regimes, and what each buys

Unstructured



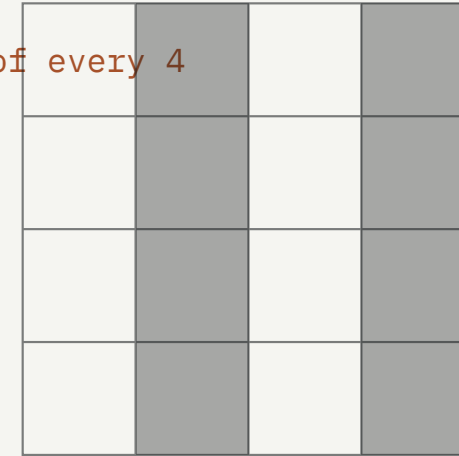
highest accuracy per weight removed
needs indices; rarely faster

2:4 semi-structured



fixed pattern, small metadata
real speedup where hardware supports it

Structured (channels)



exactly 2 of every 4

result is a smaller **dense** network
speedup everywhere, no special support

Accuracy per removed weight falls left to right. Realised speedup rises left to right. Choose by which you are short of.

Accuracy per removed weight falls left to right. **Realised speedup rises left to right.** Choose by which you are short of.

THE CLAIM, AND THE COUNTER-CLAIM

The lottery ticket hypothesis

- **Claim.** A randomly initialised dense network contains a sparse subnetwork which, trained from **the same initialisation**, matches the full network in comparable time.
- **Procedure.** Train → prune by magnitude → **reset survivors to their original initial values** → retrain → repeat.
- On MNIST and CIFAR-10 the winning tickets found were typically **10–20 % of the original size**.
- **Why it matters:** it separates two things usually conflated — whether a sparse **architecture** suffices, and whether a particular **initialisation** of it is trainable.

Read the critiques alongside it: results are sensitive to the learning-rate schedule and need **rewinding** to an early checkpoint on larger models.

DERIVATION

Break-even sparsity for storage

CSR: one value + one index per non-zero. σ = fraction SURVIVING

$$\sigma \cdot (b_v + b_i) < b_v \quad \Rightarrow \quad \sigma < b_v / (b_v + b_i)$$

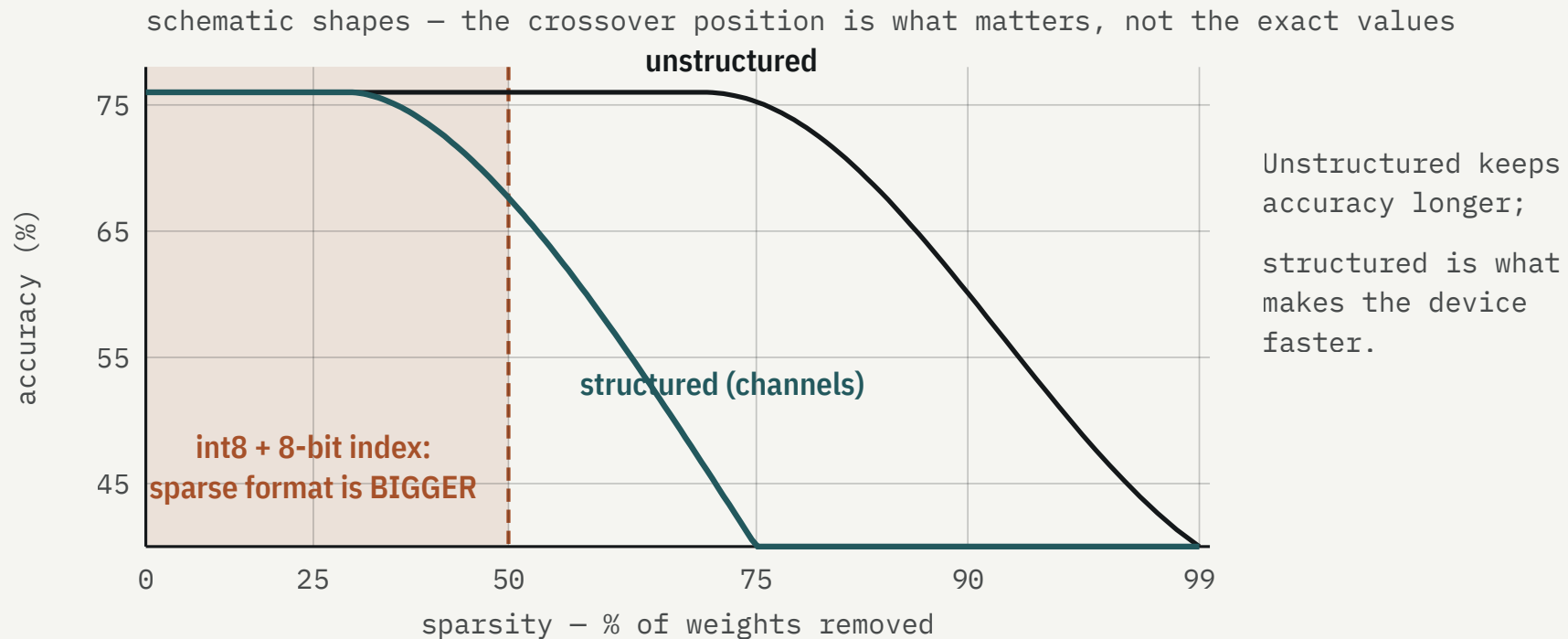
float32 + 16-bit index: $\sigma < 32/48 = 0.67 \rightarrow$ above 33 % saves

int8 + 8-bit index: $\sigma < 8/16 = 0.50 \rightarrow$ remove MORE THAN HALF

Compressing one way makes the other way harder: combining quantization with unstructured sparsity is far less attractive than combining it with structured sparsity — which is why most microcontroller deployments use structured pruning only.

AND SIZE IS THE EASY PART

Speed is harder than size



Skipping a multiply saves ≈ 0.2 pJ. The index decode, the irregular load and the lost vectorisation typically cost **more**.

THE IDEA

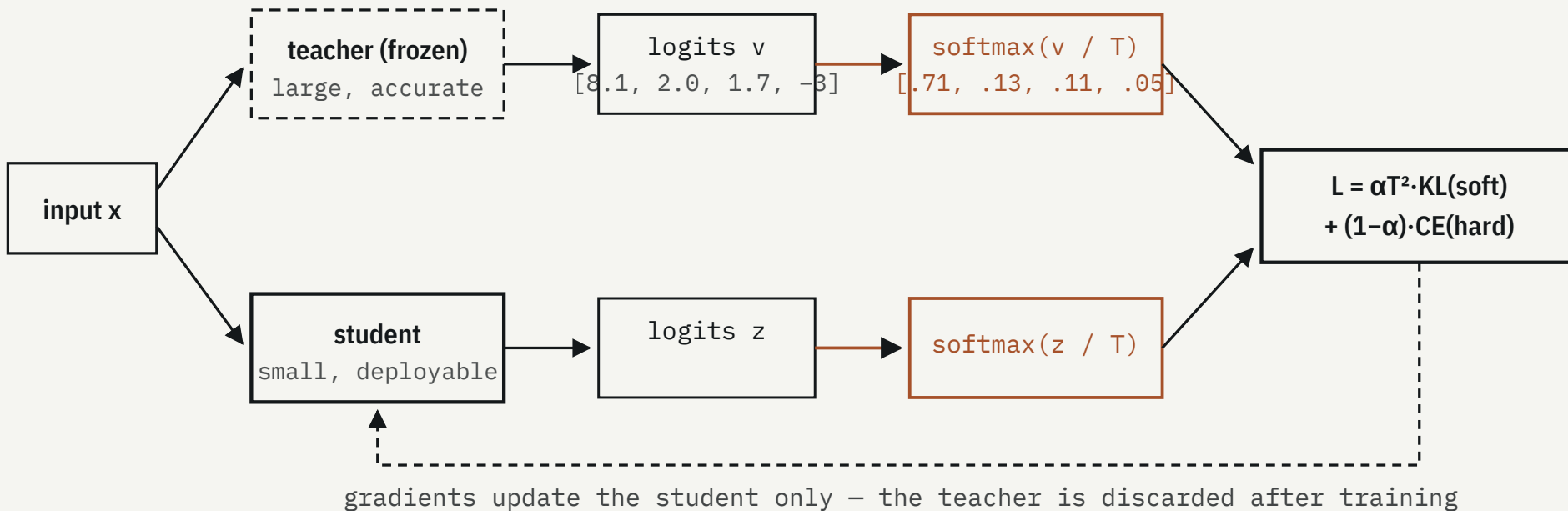
A label tells the student one bit. A teacher's softened output tells it how the whole class space is arranged.

“This is *stop* — and it is much more like *up* and *off* than like *dog*.” That relative structure over the wrong classes is what Hinton called dark knowledge, and the one-hot label never contained it.

MECHANISM

The teacher is discarded after training

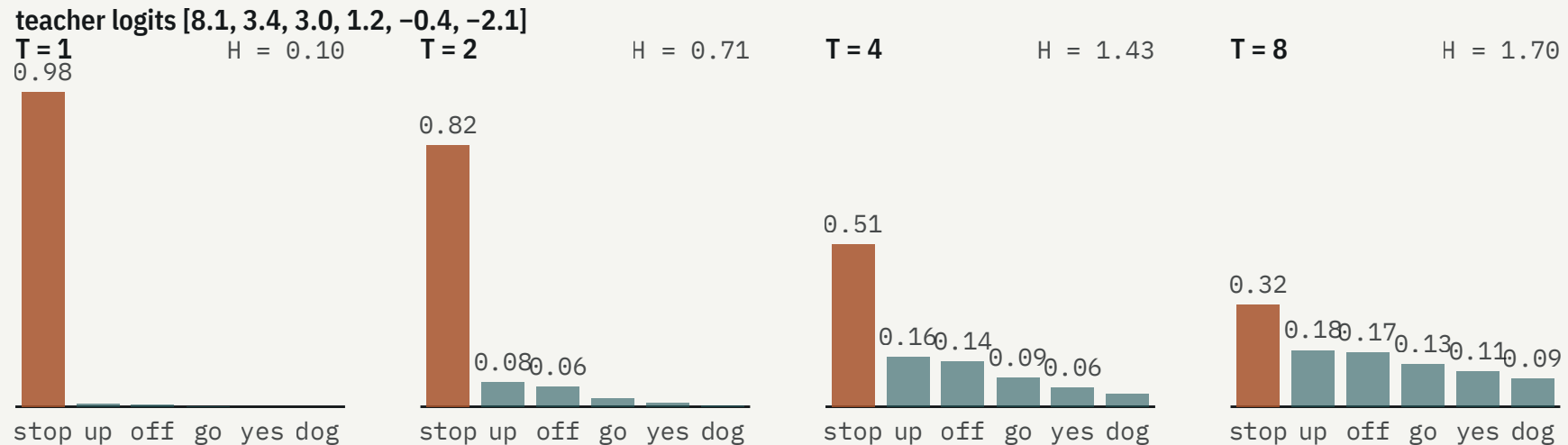
$T > 1$ flattens the distribution, revealing the ranking over wrong classes



Zero inference cost. The student ships alone.

COMPUTED ON ONE SET OF LOGITS

Where the dark knowledge lives



At $T = 1$ the teacher says only "stop". At $T = 4$ it says "stop, and it is much more like up and off than like dog" – a similarity metric the one-hot label never contained.

That extra structure is the dark knowledge. It is also why the loss needs the T^2 correction: softening shrinks the gradient by about $1/T^2$, and the factor puts it back.

Entropy H rises with T . At $T = 1$ almost nothing beyond the label; at $T = 8$ the useful ranking is washing out too.

PUTTING IT TOGETHER

A compression recipe — the baseline your method should beat

- **Fix the budget and the measurement** on the target, with the deployment toolchain.
- **Choose an efficient architecture** scaled to the budget.
- **Distil** from a float teacher while the model is still in floating point.
- **Prune channels** if latency or SRAM still binds; fine-tune with the teacher.
- **Quantize to int8 last** — PTQ, then QAT if the drop is too large.
- **Re-measure** after every step; keep the Pareto front.

BEFORE SESSION 4

Reading

B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” CVPR 2018 (§2–3); S. Williams, A. Waterman, D. Patterson, “Roofline,” *CACM* 52(4), 2009.

Why: Jacob et al. is the arithmetic of every int8 toolchain; the roofline paper is the mental model for Session 4.

Presenters in topics A–C: today's second half is your background. Re-read the matching part of the web chapter.

IF YOU REMEMBER ONE THING

Pick the model by the data and the budget; count its parameters, activations and operations; then compress in an order the hardware can cash in — int8 last, structure before scatter, distillation to recover — and check every claim on the device.

BEFORE YOU GO

Think about these

- Q1 Quantization replaces a per-value exponent with a per-tensor scale. Give a layer for which this is benign and one for which it is harmful.
- Q2 Is pruning weight selection or architecture search? State the evidence on each side.
- Q3 A paper reports 10× compression with no accuracy loss. List five questions before believing it is 10× cheaper to run.